17.2.1 Durations

Durations can be added to and subtracted from time objects:

```
now = Time.current
# => Mon, 09 Aug 2010 23:20:05 UTC +00:00
now + 1.year
#  => Tue, 09 Aug 2011 23:21:11 UTC +00:00
now - 1.week
# => Mon, 02 Aug 2010 23:21:11 UTC +00:00
```

They translate to calls to `since` or `advance`. For example here we get the correct jump in the calendar reform:

```
Time.utc(1582, 10, 3) + 5.days
# => Mon Oct 18 00:00:00 UTC 1582
```

18 Extensions to `File`

18.1 `atomic_write`

With the class method `File.atomic_write` you can write to a file in a way that will prevent any reader from seeing half-written content.

The name of the file is passed as an argument, and the method yields a file handle opened for writing. Once the block is done `atomic_write` closes the file handle and completes its job.

For example, Action Pack uses this method to write asset cache files like `all.css`:

```
File.atomic_write(joined_asset_path) do |cache|
  cache.write(join_asset_file_contents(asset_paths))
end
```

To accomplish this `atomic_write` creates a temporary file. That's the file the code in the block actually writes to. On completion, the temporary file is renamed, which is an atomic operation on POSIX systems. If the target file exists `atomic_write` overwrites it and keeps owners and permissions. However there are a few cases where `atomic_write` cannot change the file ownership or permissions, this error is caught and skipped over trusting in the user/filesystem to ensure the file is accessible to the processes that need it.

Due to the chmod operation `atomic_write` performs, if the target file has an ACL set on it this ACL will be recalculated/modified.

Note you can't append with `atomic_write`.

The auxiliary file is written in a standard directory for temporary files, but you can pass a directory of your choice as second argument.

Defined in `active_support/core_ext/file/atomic.rb`.

19 Extensions to `Marshal`

19.1 `load`

Active Support adds constant autoloading support to `load`.

For example, the file cache store deserializes this way:

```
File.open(file_name) { |f| Marshal.load(f) }
```

If the cached data refers to a constant that is unknown at that point, the autoloading mechanism is triggered and if it succeeds the deserialization is retried transparently.

If the argument is an `IO` it needs to respond to `rewind` to be able to retry. Regular files respond to `rewind`.

Defined in `active_support/core_ext/marshal.rb`.